

Performance Testing

Date	15 July 2026
Team ID	SWTID-2026-3798
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Functional Performance Testing
Target Module / API	/predict Route
Test Environment	Local Flask Server
Test Date	10 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single Prediction	1	30	Prediction generated
2	Multiple Requests	10	60	Stable performance
3	Invalid Input Test	5	30	Validation handled
4	Continuous Requests	20	120	No server crash

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.42 sec	Pass	Fast
2	Response Time (Max)	< 5 seconds	1.10 sec	Pass	Stable
3	Throughput (Req/sec)	-	24 req/sec	Pass	Good
4	Error Rate	< 1%	0.01%	Pass	No errors
5	CPU Utilization	< 80%	34%	Pass	Normal
6	Memory Utilization	< 80%	41%	Pass	Stable

Step 4: Observations & Analysis

Key Findings

Application responded quickly and generated accurate flood predictions under normal load.

Bottlenecks Identified

No major bottlenecks observed during local testing.

Optimization Steps Taken

Used a pre-trained XGBoost model with Joblib and feature scaling to reduce prediction time.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

